NEXAFORGE AI

Layered Agent Memory Architecture

Design standards for working, episodic, semantic, procedural, and user-approved memory

Document ID	Version	Owner	Classification
NF-AI-006	2.0	Agent Platform Architecture	Confidential - Internal Use

Purpose

Define a layered memory architecture for assistants and agents that must remain coherent across steps or sessions. The standard distinguishes temporary working state from durable facts, procedures, and user preferences, with explicit rules for writing, retrieval, correction, deletion, and privacy.

Keywords: agent memory, working memory, episodic memory, semantic memory, personalization, retention

Document Control

Field	Value
Status	Approved methodology baseline
Effective date	21 July 2026
Review cycle	Every 12 months or upon material change
Primary owner	Agent Platform Architecture
Related standards	Security, privacy, software delivery, incident response, and data governance standards
Supersedes	Version 1.0 concise edition

How to Use This Document

Teams should use this document as a design and review guide, not as a substitute for engineering judgment. Sections describe mandatory controls, recommended practices, decision points, and evidence expected at release. Tailoring is permitted when documented and approved by the accountable owner. Any deviation that increases risk must include compensating controls and a review date.

1. Purpose and Objectives

Define a layered memory architecture for assistants and agents that must remain coherent across steps or sessions. The standard distinguishes temporary working state from durable facts, procedures, and user preferences, with explicit rules for writing, retrieval, correction, deletion, and privacy.

2. Scope and Applicability

This standard applies to new implementations, major changes, vendor replacements, and material expansions of systems that rely on layered agent memory architecture. It covers prototypes that process real organizational data, pre-production pilots, and production services. Small experiments using synthetic data may follow a reduced process, but they must still document ownership, purpose, and data handling. The methodology does not replace legal, security, privacy, or domain-specific requirements; instead, it provides the engineering structure through which those requirements are implemented and verified.

3. Core Principles

Memory is not a transcript: Store durable, reusable information rather than every interaction.

Write only with justification: Every memory item needs a purpose, provenance, confidence, and retention policy.

Users control personal memory: Personal preferences and profile facts require clear visibility and correction paths.

Procedures are versioned knowledge: Operational instructions must be governed like code or policy.

Retrieval is contextual: Relevant memory depends on task, recency, confidence, sensitivity, and current user intent.

4. Lifecycle and Detailed Procedure

1. Memory classification

Classify candidate information as working, episodic, semantic, procedural, or personal preference memory.

- Required evidence: a reviewable artifact or trace showing that memory classification was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

2. Write decision

Apply criteria for future usefulness, stability, sensitivity, user consent, duplication, and confidence. Reject transient or speculative content.

- Required evidence: a reviewable artifact or trace showing that write decision was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

3. Normalization and provenance

Store canonical entities, timestamps, source interaction, evidence, confidence, and supersession links.

- Required evidence: a reviewable artifact or trace showing that normalization and provenance was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

4. Retention and access policy

Assign expiration, deletion rules, encryption, tenant boundaries, and who or what may retrieve the memory.

- Required evidence: a reviewable artifact or trace showing that retention and access policy was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Retrieval planning

Form a memory query from the current task. Apply filters for identity, project, time, sensitivity, and required confidence before semantic ranking.

- Required evidence: a reviewable artifact or trace showing that retrieval planning was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

6. Conflict resolution

Prefer authoritative, recent, and explicitly confirmed items. Preserve history when changes matter, and ask for confirmation when conflicts cannot be safely resolved.

- Required evidence: a reviewable artifact or trace showing that conflict resolution was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

7. Context assembly

Insert only the minimum relevant memory into the model context, labeling facts as user-confirmed, inferred, or system-derived.

- Required evidence: a reviewable artifact or trace showing that context assembly was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

8. Correction and deletion

Propagate user corrections, remove dependent inferences, update embeddings, and verify deletion across caches and replicas.

- Required evidence: a reviewable artifact or trace showing that correction and deletion was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

9. Quality review

Audit memory usefulness, stale-item rate, privacy incidents, and cases where memory changed an answer incorrectly.

- Required evidence: a reviewable artifact or trace showing that quality review was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Entry Criteria

Work may enter the methodology when a named owner, defined user outcome, initial data classification, and target environment are available. Teams should also identify downstream consumers, irreversible actions, expected traffic, and the cost of wrong answers. If these inputs are unknown, the first deliverable is a discovery brief rather than an implementation.

6. Design Review Expectations

The design review should focus on concrete decisions and evidence. Reviewers inspect architecture diagrams, state or data schemas, model and tool dependencies, evaluation plans, privacy boundaries, fallback behavior, and operating metrics. Open issues must have owners and dates. A design is not approved merely because a model can demonstrate the happy path; the team must show how the system behaves with missing inputs, conflicting evidence, unavailable services, malformed outputs, and unauthorized requests.

7. Documentation and Traceability

Each release maintains a versioned record of requirements, decisions, prompts or policies, model identifiers, data or index versions, test results, known limitations, and approvers. Traceability should allow an incident responder to determine which configuration produced a specific output. Documentation must be concise enough to maintain but detailed enough for a new engineer to reproduce critical behavior.

8. Change Management

Changes are categorized as low, medium, or high impact. Low-impact changes include wording or monitoring adjustments with no behavior change. Medium-impact changes affect routing, prompts, thresholds, or non-sensitive data. High-impact changes include model replacement, new write capabilities, new sensitive data, changed jurisdictions, or altered decision authority. Medium and high-impact changes require targeted regression testing; high-impact changes repeat the relevant risk and approval reviews.

9. Operational Readiness

Before launch, the team confirms monitoring, alert ownership, on-call procedures, rollback, rate limits, access controls, data retention, incident communication, and user support. Synthetic probes should continuously test critical paths. Runbooks should describe both technical recovery and the user-facing behavior during degradation.

10. Continuous Improvement

Production evidence is reviewed on a regular cadence. Teams analyze failures by root cause rather than by surface symptom, distinguish model errors from retrieval, tool, data, policy, or interface errors, and prioritize improvements by user harm and frequency. Confirmed defects become regression tests. Metrics are segmented by use case, language, tenant, and risk tier to avoid hiding poor performance inside global averages.

12. Roles and Responsibilities

Role	Accountability
Memory architect	Defines layers, schemas, and retrieval logic.
Privacy engineer	Defines consent, minimization, and deletion.
Product owner	Defines personalization value and user controls.
Platform engineer	Implements storage, indexing, and consistency.
Evaluation lead	Measures relevance and harmful memory influence.

13. Measurement Framework

Metric	Definition and Use
Memory precision	Retrieved items that are relevant and correct.
Memory recall	Required known facts successfully retrieved.
Stale memory rate	Retrieved items no longer valid.
Write acceptance rate	Candidate items approved for durable storage.
Correction propagation time	Time until corrected facts stop appearing.
Deletion completeness	Stores and replicas confirming deletion.
Personalization benefit	Measured improvement attributable to approved memory.

Metrics must be segmented by use case and risk tier. Teams should define a baseline, target, alert threshold, and owner for each production metric. A metric without an action rule is considered observational rather than operational.

14. Worked Example

A project assistant learns that a team uses Python 3.12 and deploys to a regulated environment. The version constraint is stored as project semantic memory with a source and review date. A temporary debugging hypothesis remains only in working memory. When the team later upgrades to Python 3.13, the new confirmed fact supersedes the old one while preserving change history for reproducibility.

15. Risk Register and Controls

Risk	Primary Controls

Storing speculation	Require confirmation or strong evidence before durable writes.
Cross-user leakage	Enforce hard tenant and identity boundaries before semantic search.
Stale personalization	Use expiry, review dates, and contradiction detection.
Over-personalization	Limit retrieval to task-relevant items and expose user controls.
Deletion gaps	Maintain deletion tombstones and run consistency checks.
Memory poisoning	Treat user and tool inputs as untrusted, scan for malicious procedural instructions, and restrict who can write shared memory.

16. Release Gate

- A named business and technical owner has approved the intended use and operating boundaries.
- Required architecture, security, privacy, and risk reviews are complete.
- Evaluation results meet minimum thresholds and no severe unresolved defect remains.
- Monitoring, alerting, rollback, and incident procedures have been tested.
- User-facing disclosures, approval checkpoints, and fallback behavior are implemented where required.
- All model, prompt, tool, data, index, and policy versions are recorded in the release manifest.

17. Review Checklist

- Is the user outcome specific, measurable, and appropriate for AI assistance?
- Are authority boundaries and prohibited actions explicit?
- Can every important output be traced to inputs, evidence, and configuration?
- Does the design handle missing, conflicting, stale, or malicious inputs?
- Are sensitive data, permissions, retention, and deletion controlled?
- Are severe failures represented in the evaluation suite?
- Can the service degrade or stop safely when dependencies fail?
- Are metrics connected to owners and response procedures?

18. Appendices

Memory item schema

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Layered Agent Memory Architecture in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Write decision rubric

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Layered Agent Memory Architecture in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Conflict resolution policy

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Layered Agent Memory Architecture in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Deletion verification checklist

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Layered Agent Memory Architecture in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Revision History

Version	Date	Summary
1.0	21 July 2026	Initial concise methodology edition.

2.0	21 July 2026	Expanded edition with detailed lifecycle, controls, roles, metrics, examples, risks, release gates, and appendices.